UNIVERSIDADE FEDERAL DE ALAGOAS — UFAL

INSTITUTO DE COMPUTAÇÃO / ENGENHARIA DE COMPUTAÇÃO

Lista de Exercícios 2 — Inteligência Artificial

Componente Curricular: ECOM031 (Graduação / PPGI)

Bruno Gomes de Barros Filho — Matrícula: 21111360

Antonio Rodrigo Lima de Andrade Tenório — Matrícula: 23210707

Professor: Evandro de Barros Costa

Maceió, AL — 2026.2

Sumário

1 Questão 1 — Representação do Conhecimento: Ontologias versus Grafos de Conhecimento **3**

1.1 Comparativo entre Sistemas Baseados em Ontologias e Grafos de Conhecimento . . . 3

1.1.1 Fundamentação Teórica, Semântica e Suposição de Mundo . . . 3

1.1.2 Mecanismos de Raciocínio e Inferência . . . 3

1.1.3 Escalabilidade e Complexidade Computacional . . . 4

1.2 Exemplo de Aplicação: Sistema de Catálogo e Recomendação Cinematográfica . . . 5

1.2.1 Abordagem por Ontologia (Modelagem Terminológica — TBox) 5

1.2.2 Abordagem por Grafo de Conhecimento (Modelagem Estrutural — LPG) . . . 5

1.3 Mapeamento de uma Ontologia para um Grafo de Conhecimento . . . 6

2 Questão 2 — Classificador dos K-Vizinhos Mais Próximos (KNN) **8**

2.1 Base de Dados e Configuração do Experimento . . . 8

2.2 As 30 Instâncias Utilizadas no Experimento . . . 8

2.3 Funcionamento do Algoritmo KNN . . . 9

2.4 Seleção do Hiperparâmetro k via Validação Cruzada Leave-One-Out (LOO-CV) . . . 10

2.5 Resultados e Análise de Desempenho 11

3 Questão 3 — Chatbot ELIZA, diálogo, algoritmo e evoluções 12
3.1 O chatbot ELIZA e o script DOCTOR 12
3.2 Trecho de diálogo e análise técnica 12
3.3 Regras e algoritmo do ELIZA 13
3.4 Resposta do ELIZA a cada sentença do diálogo 13
3.5 Evoluções: PARRY (1972) e AIML / ALICE (1995–2000) 14

1 Questão 1 — Representação do Conhecimento: Ontologias versus Grafos de Conhecimento

1.1 Comparativo entre Sistemas Baseados em Ontologias e Grafos de Conhecimento

Na área de Representação do Conhecimento e Raciocínio (RCR), a modelagem formal de domínios evoluiu a partir das redes semânticas e dos sistemas de *frames* introduzidos nas décadas de 1970 e 1980, consolidando-se em duas abordagens predominantes na literatura contemporânea: os sistemas baseados em ontologias formais e os sistemas baseados em grafos de conhecimento (*Knowledge Graphs*). Conforme apontado por Russell e Norvig (2013), qualquer formalismo de representação precisa equilibrar a expressividade axiomática necessária para inferência dedutiva e a tratabilidade computacional dos algoritmos associados.

Embora ambas as abordagens organizem entidades e relacionamentos em estruturas direcionadas, suas fundações teóricas e objetivos operacionais diferem de maneira substancial:

1.1.1 Fundamentação Teórica, Semântica e Suposição de Mundo

As ontologias formais fundamentam-se nas Lógicas de Descrição (*Description Logics* — DL), uma família de linguagens de representação baseadas em fragmentos decidíveis da Lógica de Primeira Ordem. Esse arcabouço originou o padrão W3C OWL 2 (*Web Ontology Language*), cuja semântica baseia-se na lógica $\mathcal{SROIQ}$. Numa ontologia, a semântica de classes, relações e instâncias é expressa através de axiomas rigorosos, operando sob a **Hipótese de Mundo Aberto** (*Open World Assumption* — OWA). Sob a OWA, a ausência de uma afirmação na base de dados não autoriza o sistema a deduzir sua falsidade, mas apenas indica que a informação é desconhecida. Adicionalmente, adota-se a ausência da pressuposição de nomes únicos (*Non-Unique Name Assumption*), de modo que indivíduos com identificadores distintos podem ser inferidos como equivalentes (`owl:sameAs`), salvo declaração explícita de desigualdade.

Por outro lado, os Grafos de Conhecimento surgiram da convergência prática entre a Web Semântica (especificações RDF e RDF Schema) e as tecnologias modernas de bancos de dados orientados a grafos, com ênfase no modelo de Grafos de Propriedades Rotuladas (*Labeled Property Graphs* — LPG, como o motor Neo4j). Nesse modelo, a prioridade reside na representação estruturada e na conectividade em larga escala de instâncias do mundo real. Diferentemente dos sistemas ontológicos puros, as implementações de LPG adotam em suas consultas a **Hipótese de Mundo Fechado** (*Closed World Assumption* — CWA) combinada com a pressuposição de nomes únicos: a ausência de uma aresta entre dois nós é tratada pragmaticamente como inexistência do vínculo, viabilizando avaliações eficientes de conectividade.

1.1.2 Mecanismos de Raciocínio e Inferência

Nos sistemas ontológicos, o raciocínio é estritamente dedutivo e analítico. Motores de inferência lógica especializados (*reasoners* automáticos como HermiT, Pellet e FaCT++), implementados sobre algoritmos de *Tableau*, processam o conjunto de axiomas terminológicos (TBox) e assertivos (ABox). As tarefas computacionais padrão

englobam:

- **Verificação de consistência:** atestar se a ontologia admite pelo menos um modelo formal satisfatível.
- **Subsunção e classificação taxonômica:** calcular automaticamente a hierarquia implícita de classes a partir das restrições lógicas declaradas.
- **Instanciação e realização:** determinar a pertinência de indivíduos a classes abstratas complexas.

Em contrapartida, nos Grafos de Conhecimento o mecanismo primário de inferência deixa de ser a dedução lógica axiomática e passa a ser a inferência estrutural, algorítmica ou estatística. Utilizam-se consultas baseadas em casamento de padrões de subgrafos (expressas em Cypher ou SPARQL), algoritmos clássicos de teoria dos grafos (caminhos mínimos de Dijkstra, centralidade de PageRank, coeficientes de agrupamento) e técnicas indutivas baseadas em aprendizado de máquina sobre grafos, tais como modelos de representação vetorial de entidades e relações (*Knowledge Graph Embeddings* como TransE e RotatE) e Redes Neurais em Grafo (GNNs). Tais abordagens visam à predição de elos ausentes (*link prediction*) e ao alinhamento de entidades em bases ruidosas.

1.1.3 Escalabilidade e Complexidade Computacional

O poder expressivo das Lógicas de Descrição impõe custos computacionais severos. Para a linguagem OWL 2 DL, a complexidade no pior caso de problemas de satisfatibilidade atinge ordens de NEXPTIME a 2EXPTIME, o que limita o desempenho de raciocinadores automáticos completos sobre bases contendo dezenas de milhões de indivíduos.

Em contraste, os bancos de dados em grafo orientados a propriedades foram concebidos com foco em alto desempenho de leitura, escrita e travessia sobre grafos contendo bilhões de nós e arestas. Ao renunciarem à checagem automática de axiomas complexos em tempo de execução, essas plataformas empregam estruturas físicas com indexação direta por ponteiros (*index-free adjacency*), garantindo que o custo de travessia local dependa apenas do grau dos nós envolvidos, e não do tamanho global do grafo.

A Tabela 1 resume os pontos de diferenciação entre as abordagens.

Tabela 1: Síntese comparativa entre Sistemas Baseados em Ontologias e Grafos de Conhecimento

Critério de Análise	**Sistemas Baseados em Ontologias**	**Grafos de Conhecimento (LPG/RDF)**
Fundamentação Teórica	Lógicas de Descrição ($\mathcal{SROIQ}$), Lógica de Primeira Ordem	Teoria dos Grafos, Bancos de Grafos de Propriedades
Especificações Comuns	W3C OWL 2, RDF Schema, Protégé	Labeled Property Graph (Neo4j), RDF Triplestores
Suposição de Mundo	Mundo Aberto (OWA), sem nomes únicos obrigatórios	Mundo Fechado (CWA) em consultas, nomes únicos explícitos
Natureza da Inferência	Dedução lógica formal via Tableau (HermiT, Pellet)	Travessia de padrões (Cypher), centralidade, embeddings, GNNs
Foco Funcional	Validação axiomática, consistência conceitual, subsunção	Consultas de conectividade, travessia rápida, escalabilidade
Complexidade de Escala	Elevada complexidade de raciocínio (NEXPTIME / 2EXPTIME)	Travessia em tempo quase linear por adjacência direta

1.2 Exemplo de Aplicação: Sistema de Catálogo e Recomendação Cinematográfica

Para ilustrar o contraste entre as duas concepções de representação, considere o domínio de um catálogo e sistema de recomendação cinematográfica.

1.2.1 Abordagem por Ontologia (Modelagem Terminológica — TBox)

Na abordagem ontológica, o objetivo primário é formalizar o esquema conceitual e as regras de integridade do domínio:

- **Taxonomia de Classes:** Definem-se as classes conceituais `Pessoa`, `Diretor`, `Ator` e `Filme`. Especifica-se formalmente que $\texttt{Diretor} \sqsubseteq \texttt{Pessoa}$, $\texttt{Ator} \sqsubseteq \texttt{Pessoa}$, e estabelece-se o axioma de disjunção $\texttt{Pessoa} \sqcap \texttt{Filme} \equiv \bot$.
- **Propriedades de Objeto:** Modela-se a relação binária `dirigiu`, definindo-se formalmente seu domínio como `Diretor` ($\exists \texttt{dirigiu}.\top \sqsubseteq \texttt{Diretor}$) e sua imagem como `Filme` ($\top \sqsubseteq \forall \texttt{dirigiu}.\texttt{Filme}$).
- **Raciocínio Automático:** Ao inserir na base de asserções (ABox) o fato `dirigiu(Christopher_Oppenheimer)`, o raciocinador deduz automaticamente que `Christopher_Nolan` pertence à classe `Diretor` e, por conseguinte, à classe `Pessoa`, sem necessidade de anotação manual. Se um registro inconsistente declarar que um indivíduo da classe `Filme` dirigiu outro `Filme`, o motor detecta a violação do domínio e reporta a inconsistência lógica da base.

1.2.2 Abordagem por Grafo de Conhecimento (Modelagem Estrutural — LPG)

No grafo de conhecimento, a atenção concentra-se na conectividade eficiente sobre o conjunto instanciado:

- **Estrutura de Nós:** As entidades reais são nós com rótulos e propriedades literais, como `(:Pessoa:Diretor {nome: "Steven Spielberg"})` e `(:Filme {titulo: "Jurassic Park", ano: 1993})`.
- **Arestas Tipadas:** Os vínculos são arestas direcionadas explícitas, tais como `[:DIRIGIU]`, `[:ATUOU_EM]` e `[:PERTENCE_AO_GENERO]`.
- **Mecanismo de Consulta:** Para gerar uma recomendação a um usuário interessado em `"Jurassic Park"`, o sistema executa uma consulta Cypher que recupera filmes compartilhando nós adjacentes de direção, elenco ou gênero em até dois saltos de distância:

```
MATCH (f1:Filme {titulo: "Jurassic Park"})<-[:DIRIGIU]-(d:
    Diretor)-[:DIRIGIU]->(f2:Filme)
WHERE f1 <> f2
RETURN f2.titulo, count(*) AS peso ORDER BY peso DESC
    LIMIT 5;
```

  A resposta é produzida em poucos milissegundos por meio da navegação direta na estrutura de índices em grafo, sem a sobrecarga de checagem dedutiva de modelos lógicos.

1.3 Mapeamento de uma Ontologia para um Grafo de Conhecimento

A transição de uma especificação ontológica formal (representada em OWL 2) para um Grafo de Conhecimento implementado sobre o modelo de Grafo de Propriedades Rotuladas (LPG) exige a aplicação de um conjunto sistemático de transformações estruturais:

1. **Classes Ontológicas para Rótulos de Nós (*Labels*):** Cada classe da ontologia OWL converte-se em um rótulo (*label*) aplicável aos nós do grafo. A herança taxonômica (`rdfs:subClassOf`) pode ser tratada de duas formas: (i) atribuindo múltiplos rótulos a um mesmo nó, de modo que uma instância de `Diretor` receba simultaneamente os rótulos `:Pessoa` e `:Diretor`; ou (ii) mantendo nós explícitos de metadados para as classes, interligados por arestas estruturais `[:SUBCLASSE_DE]`.
2. **Propriedades de Objeto para Arestas Direcionadas Tipadas:** As *Object Properties* de OWL, que representam relações semânticas binárias entre indivíduos de classes, são traduzidas em arestas direcionadas tipadas no LPG. A propriedade ontológica `dirigiu` (com domínio `Diretor` e imagem `Filme`) materializa-se como a aresta direcionada `[:DIRIGIU]` conectando o nó de origem ao nó de destino.
3. **Propriedades de Tipo de Dados para Atributos Internos Chave-Valor:** As *Datatype Properties* de OWL mapeiam indivíduos para valores literais (tais como `xsd:string`, `xsd:integer` ou `xsd:dateTime`). No modelo LPG, esses pares não necessitam constituir nós isolados: são armazenados diretamente como propriedades internas chave-valor no nó correspondente (exemplo: `anoLancamento: 1993` armazenado dentro do nó do filme).
4. **Axiomas e Restrições para Restrições de Integridade e Camada de Apli-**

cação: Bancos de dados de grafo comuns não possuem motores de inferência lógica embutidos para avaliar axiomas em tempo de execução. Logo, axiomas de cardinalidade mínima/máxima e restrições de unicidade (`owl:hasKey`) são implementados como restrições de esquema no banco (ex.: `CREATE CONSTRAINT FOR (f:Filme) REQUIRE f.id IS UNIQUE`). Regras lógicas mais complexas (como disjunções e propriedades transitivas) são transferidas para a camada de serviço da aplicação antes da persistência.

A Figura 1 ilustra graficamente o esquema de mapeamento proposto.

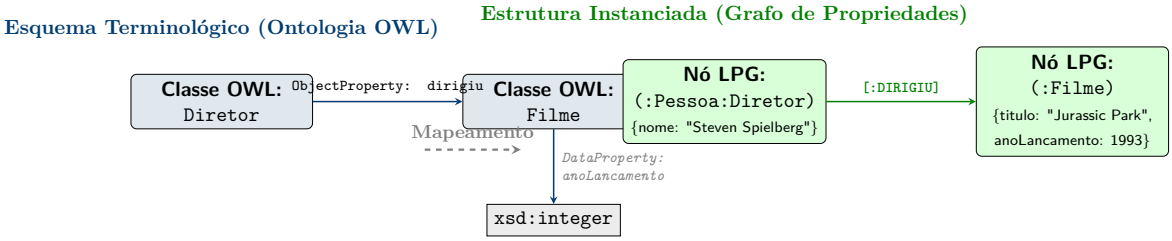


Figura 1: Mapeamento estrutural do modelo ontológico formal para grafo de propriedades rotuladas.

2 Questão 2 — Classificador dos K-Vizinhos Mais Próximos (KNN)

2.1 Base de Dados e Configuração do Experimento

O algoritmo dos K-Vizinhos Mais Próximos (*K-Nearest Neighbors* — KNN) foi avaliado sobre as mesmas 30 instâncias de risco de crédito usadas na Lista de Exercícios 1. A base parte do exemplo clássico do livro de Luger e foi ampliada na Questão 1 daquela lista para 6 atributos categóricos e 30 exemplos (E1 a E30).

Os atributos preditores são História de Crédito, Dívida, Garantia, Renda, Emprego e Imóvel. A variável-alvo é o risco de crédito, com três classes:

- **Alto**
- **Moderado**
- **Baixo**

Como o KNN trabalha com distância numérica e todos os atributos são categóricos, cada atributo foi convertido em colunas binárias por *one-hot encoding* (`get_dummies`), a mesma estratégia usada no CART da Lista 1. O espaço resultante tem 14 dimensões com valores em $\{0, 1\}$, de modo que a distância euclidiana entre dois exemplos conta, na prática, quantas categorias diferem. Não foi necessária normalização min-max.

2.2 As 30 Instâncias Utilizadas no Experimento

A Tabela 2 reproduz a base completa da Lista 1.

Tabela 2: As 30 instâncias de risco de crédito da Lista 1

Ex.	História	Dívida	Garantia	Renda	Emprego	Imóvel	Risco
E1	Ruim	Alta	Nenhuma	0–15	Instável	Alugado	Alto
E2	Desconhecida	Alta	Nenhuma	15–35	Instável	Alugado	Alto
E3	Desconhecida	Baixa	Nenhuma	15–35	Estável	Alugado	Moderado
E4	Desconhecida	Baixa	Nenhuma	0–15	Instável	Alugado	Alto
E5	Desconhecida	Baixa	Nenhuma	>35	Estável	Próprio	Baixo
E6	Desconhecida	Baixa	Adequada	>35	Estável	Próprio	Baixo
E7	Ruim	Baixa	Nenhuma	0–15	Instável	Alugado	Alto
E8	Ruim	Baixa	Adequada	>35	Estável	Alugado	Moderado
E9	Boa	Baixa	Nenhuma	>35	Estável	Próprio	Baixo
E10	Boa	Alta	Adequada	>35	Estável	Próprio	Baixo
E11	Boa	Alta	Nenhuma	0–15	Instável	Alugado	Alto
E12	Boa	Alta	Nenhuma	15–35	Estável	Alugado	Moderado
E13	Boa	Alta	Nenhuma	>35	Estável	Alugado	Baixo
E14	Ruim	Alta	Nenhuma	15–35	Instável	Alugado	Alto
E15	Boa	Baixa	Adequada	15–35	Estável	Próprio	Baixo
E16	Boa	Baixa	Nenhuma	15–35	Estável	Próprio	Baixo
E17	Desconhecida	Baixa	Adequada	>35	Instável	Próprio	Moderado
E18	Ruim	Alta	Nenhuma	>35	Estável	Alugado	Moderado
E19	Desconhecida	Alta	Nenhuma	0–15	Estável	Alugado	Alto
E20	Boa	Baixa	Adequada	>35	Estável	Alugado	Baixo
E21	Ruim	Baixa	Nenhuma	15–35	Estável	Alugado	Moderado
E22	Boa	Alta	Adequada	15–35	Estável	Próprio	Moderado
E23	Ruim	Alta	Adequada	0–15	Instável	Próprio	Alto
E24	Desconhecida	Baixa	Nenhuma	15–35	Instável	Próprio	Moderado
E25	Boa	Baixa	Nenhuma	0–15	Estável	Próprio	Moderado
E26	Ruim	Baixa	Nenhuma	>35	Instável	Alugado	Alto
E27	Desconhecida	Alta	Adequada	>35	Estável	Próprio	Baixo
E28	Ruim	Alta	Nenhuma	15–35	Estável	Alugado	Alto
E29	Boa	Baixa	Nenhuma	>35	Instável	Próprio	Baixo
E30	Desconhecida	Baixa	Nenhuma	0–15	Estável	Alugado	Alto

A distribuição ficou com 11 exemplos de risco **Alto**, 9 de **Moderado** e 10 de **Baixo**.

2.3 Funcionamento do Algoritmo KNN

O KNN constitui um algoritmo de aprendizado baseado em instâncias (*lazy learner*), dispensando a construção explícita de uma função de generalização durante a fase de treino. O processamento ocorre integralmente no momento da predição:

1. **Cálculo de Distância:** Para uma instância de teste $\mathbf{q} \in \mathbb{R}^p$, computa-se a distância euclidiana para cada ponto $\mathbf{x}_i$ do conjunto de treino:

$$d(\mathbf{q}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{p} (q_j - x_{ij})^2} \tag{1}$$

2. **Seleção dos k Vizinhos:** Ordenam-se as distâncias calculadas em ordem ascendente e isolam-se os índices dos k pontos com as menores distâncias:

$$\mathcal{N}_k(\mathbf{q}) = \arg\min_{S \subset D, |S|=k} \sum_{\mathbf{x}_i \in S} d(\mathbf{q}, \mathbf{x}_i) \tag{2}$$

3. **Regra de Decisão por Voto Majoritário:** A classe predita $\hat{y}$ corresponde à

classe com maior frequência absoluta entre os k vizinhos:

$$\hat{y} = \arg\max_{c \in C} \sum_{\mathbf{x}_i \in \mathcal{N}_k(\mathbf{q})} \mathbb{I}(y_i = c) \tag{3}$$

onde $\mathbb{I}(\cdot)$ representa a função indicadora. Em caso de empate, utilizam-se critérios determinísticos de desempate pela menor distância média ou classe majoritária a priori.

2.4 Seleção do Hiperparâmetro k via Validação Cruzada Leave-One-Out (LOO-CV)

Dado o tamanho reduzido da amostra ($N = 30$), a técnica mais adequada para seleção do hiperparâmetro k é a validação cruzada *Leave-One-Out* (LOO-CV). Nesse procedimento, cada uma das 30 instâncias é utilizada individualmente como conjunto de teste, enquanto as outras 29 instâncias atuam como conjunto de treino, repetindo-se o ciclo 30 vezes.

Foram avaliados valores ímpares $k \in \{1, 3, 5, 7, 9\}$ para minimizar a ocorrência de empates. A Tabela 3 e a Figura 2 detalham os resultados observados.

Tabela 3: Desempenho da Validação Cruzada Leave-One-Out por valor de k

Valor de k	**Acertos (/30)**	**Acurácia LOO-CV (%)**
$k = 1$	18	60,00%
$k = 3$	19	63,33%
$\mathbf{k = 5}$	**21**	**70,00%**
$k = 7$	19	63,33%
$k = 9$	18	60,00%

O melhor valor foi $\mathbf{k = 5}$, com 21 acertos em 30 (70,00%). Com $k = 1$ o classificador fica sensível a vizinhos isolados; com $k \geq 7$ a votação começa a misturar exemplos de classes vizinhas e a acurácia cai de novo.

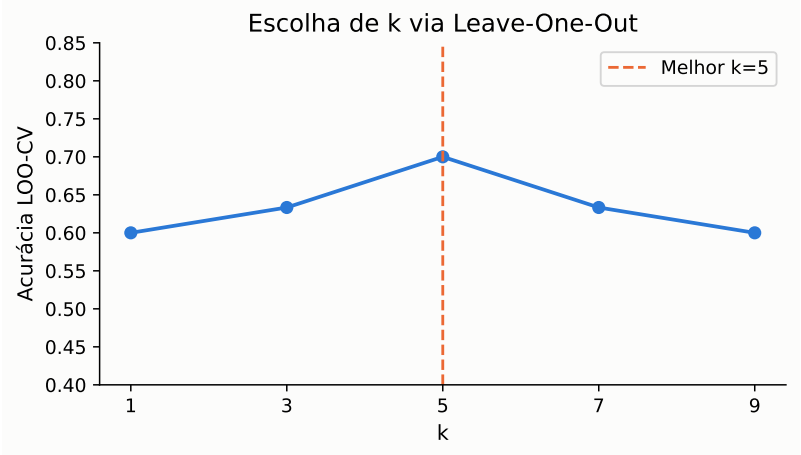


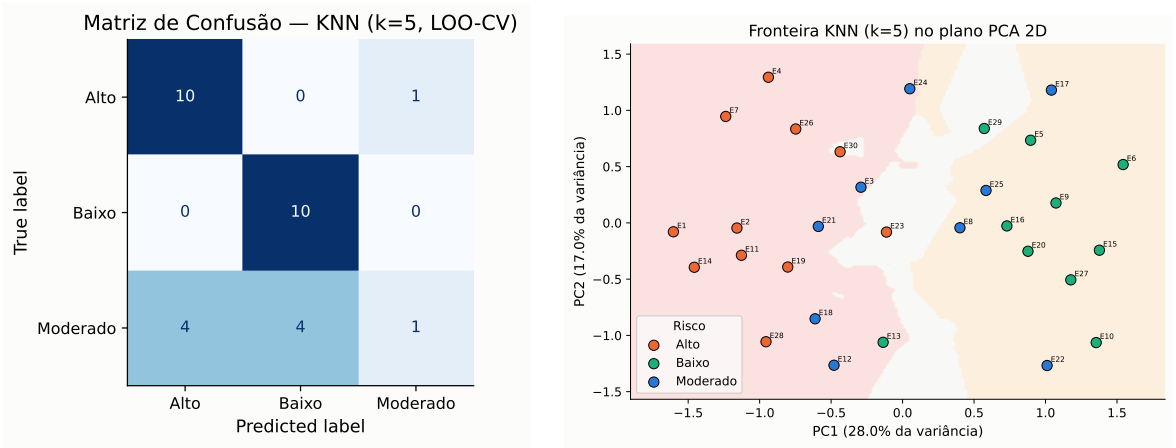
Figura 2: Curva de acurácia da validação cruzada Leave-One-Out (LOO-CV) em função de k.

2.5 Resultados e Análise de Desempenho

Com $k = 5$, as predições Leave-One-Out geram a matriz de confusão e as métricas da Tabela 4 e da Figura 3a.

Tabela 4: Métricas LOO-CV do KNN com $k = 5$ sobre as 30 instâncias

Classe	**Precisão**	**Revocação**	F_1**-Score**	**Suporte**
Alto	0,71	0,91	0,80	11
Baixo	0,71	1,00	0,83	10
Moderado	0,50	0,11	0,18	9
Acurácia Global		**70,00%**		**30**
Média Macro	0,64	0,67	0,61	30
Média Ponderada	0,65	0,70	0,63	30



(a) Matriz de Confusão ($k = 5$)

(b) Fronteira projetada em PCA 2D

Figura 3: Visualizações do KNN ($k = 5$): matriz de confusão em LOO-CV e fronteira no plano das duas primeiras componentes principais.

O classificador acerta bem as classes extremas: recupera 10 dos 11 exemplos `Alto` e todos os 10 `Baixo`. A dificuldade fica em `Moderado`, que no Leave-One-Out é confundida com Alto ou Baixo (apenas 1 acerto em 9). Isso faz sentido na base: o risco moderado ocupa uma região intermediária do espaço de atributos e, com poucos exemplos, os vizinhos tendem a puxar o voto para uma das extremidades.

Como exemplo passo a passo, o caso E1 (risco real Alto) tem como cinco vizinhos mais próximos E7, E11, E14, E2 e E4, todos da classe Alto. O voto é unânime e a predição coincide com o rótulo verdadeiro.

3 Questão 3 — Chatbot ELIZA, diálogo, algoritmo e evoluções

3.1 O chatbot ELIZA e o script DOCTOR

O ELIZA foi desenvolvido entre 1964 e 1966 por Joseph Weizenbaum no MIT Artificial Intelligence Laboratory [5]. O programa conversa por casamento de padrões e transformação de texto. Weizenbaum não pretendia modelar compreensão semântica: queria mostrar que regras simples bastavam para o interlocutor humano projetar inteligência e empatia no sistema — o chamado *efeito ELIZA*.

O script mais famoso é o DOCTOR, que emula um psicoterapeuta rogeriano (abordagem centrada na pessoa, de Carl Rogers). Nesse estilo, o terapeuta evita opinar, diagnosticar ou trazer temas externos; reformula o que o paciente disse e devolve a fala em pergunta ou comentário reflexivo. O ELIZA se aproveita exatamente desse domínio reduzido.

Principais funcionalidades.

1. **Casamento de padrões:** a sentença é decomposta a partir de palavras-chave e curingas.
2. **Remontagem:** trechos capturados entram em moldes de resposta, com troca de pessoa gramatical (`I`/`me`/`my` → `you`/`your`).
3. **Pilha de palavras-chave (*keystack*):** cada palavra-chave tem um peso; a de maior prioridade é processada primeiro.
4. **Memória de tópicos:** algumas declarações (em especial com `my`) são guardadas para reaparecer se a conversa estagnar.
5. **Respostas de contingência:** sem palavra-chave reconhecida, o sistema usa frases neutras (*"Please go on"*, *"What does that suggest to you?"*).

3.2 Trecho de diálogo e análise técnica

O diálogo abaixo adapta, em português, a interação clássica de Weizenbaum (1966) com o script DOCTOR.

```
[U1] Homens são todos iguais.
[E1] DE QUE MANEIRA?
[U2] Eles estão sempre nos incomodando por causa de alguma coisa.
[E2] VOCÊ CONSEGUE PENSAR EM UM EXEMPLO ESPECÍFICO?
[U3] Bem, meu namorado me fez vir aqui.
[E3] SEU NAMORADO FEZ VOCÊ VIR AQUI?
[U4] Ele disse que eu fico deprimida quase o tempo todo.
[E4] SINTO MUITO EM OUVIR QUE VOCÊ FICA DEPRIMIDA.
[U5] É verdade. Eu sou infeliz.
[E5] VOCÊ ACHA QUE VIR AQUI VAI TE AJUDAR A NÃO SER INFELIZ?
[U6] Eu preciso de um pouco de ajuda, isso é certo.
[E6] O QUE SIGNIFICARIA PARA VOCÊ SE VOCÊ CONSEGUISSE AJUDA?
[U7] Talvez eu pudesse aprender a me dar bem com minha mãe.
[E7] FALE-ME MAIS SOBRE A SUA FAMÍLIA.
[U8] Você não gosta de mim.
[E8] O QUE FAZ VOCÊ PENSAR QUE EU NÃO GOSTO DE VOCÊ?
```

```
[U9] Você não é uma pessoa de verdade, você é um computador.
[E9] IMPORTA SE EU SOU UM COMPUTADOR?
```

Partes tecnicamente interessantes. Em [U3]→[E3], a palavra-chave ligada a `meu/my` captura o segmento possessivo; a troca pronominal e a remontagem interrogativa geram *"Seu namorado fez você vir aqui?"*. Em [U4]→[E4], o estado *deprimida* dispara um molde de empatia. Em [U7]→[E7], `mãe` é generalizada para o tópico *família*, em vez de ecoar a frase inteira. Em [U2]→[E2], `sempre/always` pede um exemplo concreto — típico do estilo rogeriano.

Partes tecnicamente falhas. Não há memória semântica entre turnos: cada sentença é quase isolada. Negações e escopo lógico confundem o casamento ("não estou infeliz" ainda pode acionar a regra de *infeliz*). Em [U9], a exposição do truque ("você é um computador") só recebe um molde de negação de pressuposto. Em [E5], a pergunta sobre "vir aqui" presume continuidade terapêutica que [U5] não afirmou: o molde veio da palavra-chave, não de um plano de diálogo.

3.3 Regras e algoritmo do ELIZA

O motor é determinístico e opera em fases: (1) normalização e tokenização; (2) varredura de palavras-chave e montagem da *keystack*; (3) casamento com regras de decomposição; (4) substituição pronominal; (5) remontagem no próximo molde da lista circular. Se nenhuma regra da palavra-chave atual casar, o algoritmo desce na pilha; se a pilha acabar, usa memória de tópicos ou *fallback*.

Exemplo clássico para a palavra-chave `I REMEMBER`: o padrão `(* I REMEMBER *)` decompõe *"Well, I remember when we lived in Maceió"* em três partes; o molde `(DO YOU OFTEN THINK OF (3) ?)` produz *"DO YOU OFTEN THINK OF WHEN YOU LIVED IN MACEIO?"*, após a troca pronominal nos segmentos capturados.

3.4 Resposta do ELIZA a cada sentença do diálogo

A Tabela 5 liga cada turno do usuário à palavra-chave dominante, à transformação e à saída (versão em português do trecho acima). Em todos os casos o "entendimento" é ilusório: o sistema ranqueia palavras-chave, decompõe, troca pronomes e preenche um molde.

Tabela 5: Mapeamento turno a turno do diálogo com o script DOCTOR.

Turno	**Gatilho**	**Transformação**	**Saída**
U1→E1	generalização	molde interrogativo fixo	De que maneira?
U2→E2	`sempre/always`	pede concretude	Exemplo específico?
U3→E3	`meu/my`	captura + troca pronominal	Seu namorado fez você vir aqui?
U4→E4	estado (*deprimida*)	molde de empatia	Sinto muito. . . deprimida.
U5→E5	*infeliz*	molde sobre ajuda	. . . ajudar a não ser infeliz?
U6→E6	*preciso/ajuda*	pergunta hipotética	O que significaria. . . ajuda?
U7→E7	`mãe`→família	generaliza o tópico	Fale-me mais sobre a sua família.
U8→E8	acusação ao terapeuta	inverte em pergunta	O que faz você pensar. . . ?
U9→E9	exposição (*computador*)	negação de pressuposto	Importa se eu sou um computador?

3.5 Evoluções: PARRY (1972) e AIML / ALICE (1995–2000)

PARRY (Kenneth Colby, Stanford, 1972). O PARRY simula o discurso de uma pessoa com esquizofrenia paranoica [6]. Diferente do ELIZA, mantém um modelo de personalidade com variáveis afetivas contínuas — medo (*fear*), raiva (*anger*) e desconfiança (*mistrust*). A mesma pergunta do interlocutor gera respostas distintas conforme esses níveis. Com medo e raiva baixos, o sistema pode falar de suas obsessões; com níveis altos, fica hostil ou evasivo. Em avaliações com psiquiatras e transcrições cegas, os médicos nem sempre distinguiram o programa de pacientes reais. O salto em relação ao ELIZA é a introdução de **estado interno** que altera a seleção das respostas.

AIML e A.L.I.C.E. (Richard Wallace, 1995–2000). A linguagem AIML (*Artificial Intelligence Markup Language*) e o bot A.L.I.C.E. (*Artificial Linguistic Internet Computer Entity*) padronizam a base conversacional em categorias XML [7]. Há normalização recursiva com a tag `<srai>`, contexto com `<that>` e `<topic>`, e indexação em árvore (*Graphmaster*), o que permite dezenas de milhares de regras com busca rápida. A.L.I.C.E. venceu o Prêmio Loebner em mais de uma edição. Em relação ao ELIZA, o ganho principal é escala, modularidade e controle explícito de contexto — ainda sem compreensão semântica profunda.

Tabela 6: Comparativo das três gerações de chatbots clássicos.

Critério	**ELIZA (1966)**	**PARRY (1972)**	**AIML/ALICE (1995)**
Arquitetura	script + padrões	regras + estados afetivos	categorias XML (Graphmaster)
Modelo interno	quase nenhum	variáveis emocionais	base hierárquica de regras
Contexto	pilha de tópicos	filtrado pelo estresse	`<that>`, `<topic>`
Variações	decomposição rígida	heurísticas de intenção	recursão `<srai>`
Domínio	escuta clínica	paranoia simulada	cobertura conversacional ampla

A linha evolutiva é clara: o ELIZA mostra que padrões sintáticos bastam para a ilusão conversacional; o PARRY acrescenta estado afetivo; o AIML/ALICE transforma o acervo de regras num formalismo modular e indexável.

Referências

[1] RUSSELL, Stuart J.; NORVIG, Peter. *Inteligência Artificial: uma abordagem moderna*. 3. ed. Rio de Janeiro: Elsevier / Campus, 2013.

[2] CARVALHO, André C. P. L. F. et al. *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. Rio de Janeiro: LTC, 2011.

[3] LUGER, George F. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*. 6. ed. Boston: Pearson, 2009.

[4] TENÓRIO, Antonio Rodrigo L. A.; GOMES, Bruno B. F. *Lista de Exercícios 1 — Inteligência Artificial*. Notebook `Lista_IA.ipynb`, UFAL, 2026.

[5] WEIZENBAUM, Joseph. ELIZA — a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, v. 9, n. 1, p. 36–45, 1966.

[6] COLBY, Kenneth M.; WEBER, Sylvia; HILF, Franklin D. Artificial paranoia. *Artificial Intelligence*, v. 2, n. 1, p. 1–25, 1971/1972.

[7] WALLACE, Richard S. The anatomy of A.L.I.C.E. In: EPSTEIN, R.; ROBERTS, G.; BEBER, G. (eds.). *Parsing the Turing Test*. Dordrecht: Springer, 2009. p. 181–210.